Contriviewer User Guide

Contriviewer records editing activity in open projects and shows User, Assisted, and Agent ratios by saved session, file, and Git commit. Categories are estimates based on the editing information it can establish.

1. Install and start

You need **VS Code 1.102.0 or later** and the distributed **Contriviewer VSIX file** (contriviewer-<version>.vsix).

1. Open the Command Palette: Cmd+Shift+P on macOS or Ctrl+Shift+P on Windows/Linux.
2. Run **Extensions: Install from VSIX...** and select the VSIX file. Reload the window if prompted.
3. Open your project with **File > Open Folder...**. Monitoring starts automatically and covers every open folder in a multi-root workspace.
4. Select the **Contriviewer** icon on the left to open **Overview**. Edit and save files as usual to update the view.

For commit-level summaries, set up the project's Git repository and your Git user name and email.

2. Read the values

The top **Edit Sessions** panel counts one session per edited file when you save it, including auto-save. Saving several edited files counts their sessions separately. Saving without changes, or undoing all changes back to the previous saved state before saving, adds no session or change volume.

Category	Meaning
User	Basic editing such as typing, deletion, Cut and Paste. Automatic bracket/quote pairing at the cursor is included.
Assisted	Changes classified as using completion, snippets, refactoring, formatting or similar tools
Agent	Changes confirmed as applied to files through work delegated to Copilot, Codex or Claude
Unknown	Changes whose editing origin could not be established. A session is Unknown when no other editing category could be established.

View item	What it shows
Edit Sessions	Total Sessions and each category's count and percentage
Changed Characters	Inserted + deleted character counts accumulated from saved sessions, with category percentages
Retained File Contribution	Category ratios for text currently remaining in each file. Expand a file for character counts.
Commit Contribution	Contribution of actual inserted/deleted commit characters with observed attribution. Shows category ratios and total changed characters; expand for per-file values.

In file contribution, **Others** combines **Other Contributor** (text attributed to another Git contributor) and **External** (text found through file or Git state without directly observing its editing actions).

A session using tool assistance or combining User and Agent changes is **Assisted**. For example, if an Agent applies code and you edit the same file before saving once, that is one Assisted session; its character counts remain divided between User and Agent. Sessions use established editing categories even when some

changes are Unknown. Character ratios include Unknown.

Sessions compare **counts**, change volume compares **inserted/deleted characters**, and file contribution compares **remaining characters**, so their ratios can differ. Paste is treated as basic User editing regardless of who originally wrote the copied code.

Undo/Redo counts as a User action while restored characters keep their original source.

The **U / A / Ag** status-bar values, plus **?** (Unknown) when present, repeat the top session ratios; select them to open Overview.

Earlier history marks previously separate records and the file state recorded at that time. They remain available alongside current records without overwriting their originals.

Verified Codex and Claude changes can also appear as Agent while the project is open in VS Code. Saving files outside VS Code does not add saved edit sessions, so the top ratios and retained-file contribution can differ. Work completed before collection started is not counted as new Agent editing.

When related change information arrives later, earlier classifications and ratios may be refined. Completed Export files stay unchanged; a new Export includes the refined results.

3. Buttons and commands

Files show **5 per page, most recently updated first**; commits show **3 per page, newest commit date first**. Enter a file name/path or part of a commit hash and press **Search** or Enter. Clear and submit the search to return to all items. Use **Previous / Next** for more results. Expand a commit, then **Files**, for file details.

File lists show only files with a positive User, Assisted, Agent or Unknown character count. Retained files use remaining characters; a commit's Files use its changed characters. Hidden records are preserved and included in Export.

The branch filter uses the branch where each commit was first observed; it does not switch Git branches. The list is empty until commits have been collected.

Type Contriviewer in the Command Palette to find these commands.

Command	Function / view shortcut
Show Overview	Open Overview, also available from the left-hand icon or status bar.
Select Contribution Branch	Choose the displayed commit branch, also available through **Filter branch**.
Refresh Git Contribution	Recheck the current Git state and new commits.
Export Collected Data	Export your logs, also available through **Export collected data**.

4. Export your logs

1. Save the files you are editing.
2. Select **Export collected data** in Overview.
3. Choose a folder and filename in the Save dialog, then select **Export**. Canceling creates no file. Choose another name if the file already exists.
4. Wait for the progress notification to finish and **Export complete** to appear, then find the .tar.gz file at your selected location. Submit this file as it is.

Every Export contains the **full history from monitoring start through that Export**, so you do not need to combine earlier files. Existing records and the project's archive original are retained.

When separate histories appear together, one Export file contains their complete individual archive bundles.

Exports include edited text. Saving outside your Git repository helps avoid accidentally committing the archive.

5. Practical checks

- **Files covered:** supported source files such as Java, JavaScript, TypeScript and Python, configuration such as build.gradle, documents such as README.md, and files such as .gitignore. Coverage does not include every text file; ordinary .txt files are excluded.
- **Excluded folders:** build, dependency and resource folders such as build, dist, node_modules, assets, resources and public, and all dot-prefixed directories within the workspace. Supported dotfiles outside those directories remain included. Images and other unsupported files are excluded.
- **Concurrent use:** editing the same project in several VS Code windows can cause conflicting or missing records.
- **Sensitive information:** do not enter real passwords, API keys, or personal information into monitored files; use example values or placeholders instead. Text that was never committed to Git, or was later deleted or undone, may still remain in edit records and Exports.

6. Privacy and stored data

- **Storage:** new records normally use the project's .vscode/contriviewer/data/ and continue whether opened as a folder or through .code-workspace. Opening an inner folder reuses existing ancestor records; logs are not created throughout the folder tree. Older records remain at their original location and are linked automatically. Completed Export originals remain in the recording location's .vscode/contriviewer/ area.
- **Retention and reinstallation:** records do not expire. Updates and reinstalls reuse compatible data at the same location. Preserve the data folder; do not manually edit or delete it.
- **Recorded content:** edit times, inserted/deleted text, category summaries and Git contribution. File contents needed to compare Git changes are also stored locally. Local Git summaries also contain Git names/emails and commit messages.
- **Agent changes:** the extension reads related local Codex/Claude records. Original and modified code, and Copilot edit proposals, may appear in logs and Exports, including code you did not apply. Full conversations and prompts are not separately stored.
- **Clipboard:** the extension may temporarily read clipboard text to check a paste operation. It does not keep a separate clipboard history, but text pasted into a document is included in edit records.
- **Export and privacy:** editing records, summaries, branch names and diagnostics/recovery material are included. Edited text, Agent proposal code and diagnostics/recovery may reveal identities or paths, so do not treat an Export as an anonymized file.
- **Transfer and sharing:** no automatic uploads. Export creates a local file. Do not publish logs/Exports in public repositories or forums. Your chosen Export destination is not automatically ignored by Git; check included files before committing.

7. Troubleshooting

Symptom	Checks and action
Icon or commands are missing	Check installation/enabling in Extensions and whether VS Code is in Restricted Mode, then use **Developer: Reload Window** if needed.
Editing does not update counts or ratios	Check that the file belongs to the open project and is a covered file, then save it. Files in excluded folders are not counted. Reload the window if the view still does not update.
Git commits are missing	Check the Git repository and your Git user name/email settings. Check the hash search and **Filter branch**, then run **Refresh Git Contribution**.
Agent changes do not appear immediately	Check that the project was open during the work, save the file and allow time for an update. Ratios may be refined when related information becomes available.
Codex/Claude changes remain External	Work with the project open in an environment where the tool's local records are available. Missing records or work completed before collection may leave changes External.
Unknown appears	Available information cannot distinguish the editing category. Records are preserved and may be refined later. External instead means text whose editing actions were not directly observed.
Paste classification differs from expectations	Check **Editor: Paste As Enabled**. Disabling it, using **Paste As...**, or using custom paste commands can limit Paste observation.
History is missing after an update or reinstall	Open the same project or .code-workspace and check that its data folder remains. If an unsupported-data-format message appears, preserve the data and contact support. In this case, collection stops without overwriting existing data.
Recording unavailable / Recording needs attention	Open **Details**, resolve write permissions or free-space problems at the recording location, then select **Reload Window**. New activity is not recorded while startup has failed. Do not manually delete logs.
Export fails	Check write permissions and free space at both the project and selected destination. Use a new name if the file exists, retry, and wait for **Export complete**. A failed copy preserves the completed internal original and existing records.

If problems persist, preserve the data and send the distributor the **error message, extension/VS Code versions and what happened**. Never post entire logs publicly.